

ANS	Projekt	Akademia Nauk Stosowanych w Nowym Sączu	
	Programowanie współbieżne i rozproszone	Wydział Nauk Inżynieryjnych	
Projekt nr: P3	Temat: Klasyczne problemy wielowątkowości.	Data wykonania: 20.03.2026	Data oddania: 21.03.2026
		Prowadzący: mgr inż. Daniel Drozd	
Wykonał: Henryk Szambelan		Ocena:	
Numer albumu: 33839			
Kierunek: INS	Grupa projektowa: P NST		

I. Cel i zakres zajęć

Celem zajęć jest rozpoznawanie i eliminowanie klasycznych błędów pojawiających się w programach wielowątkowych. Poznamy mechanizmy synchronizacji — mutexy i semaforey — oraz nauczymy się ich poprawnego stosowania.

Po ukończeniu zajęć student powinien:

- rozumieć pojęcia: race condition, deadlock, starvation,
- poprawnie używać mutexów (`pthread_mutex_t`) do ochrony sekcji krytycznych,
- rozpoznawać i unikać zakleszczeń (deadlock),
- używać semaforów (`sem_t`) do synchronizacji wątków,
- analizować kod pod kątem błędów współbieżności.

II. Zadania

Zadanie 1

1. Wyniki 5 uruchomień programu `race_bad.c`

Poniższa tabela prezentuje wyniki pięciu kolejnych uruchomień programu bez mechanizmów synchronizacji (wartość oczekiwana to 0).

Uruchomienie	Wynik końcowy zmiennej
1	1583066
2	1272941
3	1388531
4	1736071
5	1272280

Tabela 1: Wyniki programu wykazującego zjawisko wyścigu.

2. Kod źródłowy zmodyfikowanej wersji z inkrementacją i dekrementacją

W poniższej wersji zastosowano muteks (pthread_mutex_t), aby zapewnić wyłączny dostęp do sekcji krytycznej podczas operacji inkrementacji i dekrementacji. Program tworzy 4 wątki, z których każdy wykonuje po milion iteracji w jednej wspólnej funkcji modyfikuj.

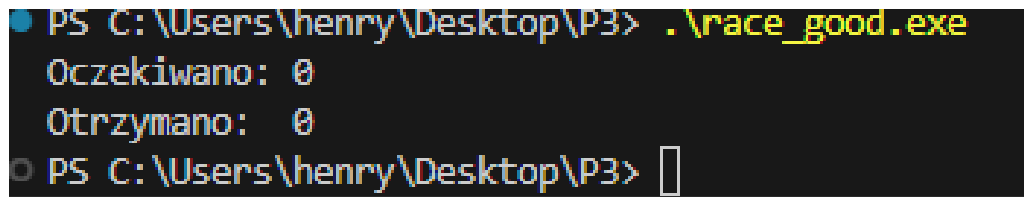
```
1  #include <pthread.h>
2  #include <stdio.h>
3
4  #define WATKI      4
5  #define ITERACJE 1000000
6
7  long licznik = 0;
8  pthread_mutex_t mutex; // mutex chroniaczy licznik
9
10 void* modyfikuj(void* arg) {
11     for (int i = 0; i < ITERACJE; i++) {
12         // Sekcja krytyczna - inkrementacja
13         pthread_mutex_lock(&mutex);
14         licznik++;
15         pthread_mutex_unlock(&mutex);
16
17         // Sekcja krytyczna - dekrementacja
18         pthread_mutex_lock(&mutex);
19         licznik--;
20         pthread_mutex_unlock(&mutex);
21     }
22     return NULL;
23 }
24
25 int main(void) {
26     pthread_mutex_init(&mutex, NULL); // inicjalizuj mutex przed
        uzyciem!
27
28     pthread_t tids[WATKI];
29     for (int i = 0; i < WATKI; i++)
30         pthread_create(&tids[i], NULL, modyfikuj, NULL);
31
32     for (int i = 0; i < WATKI; i++)
33         pthread_join(tids[i], NULL);
34
35     // Skoro kazdy watek robi tyle samo operacji ++ co --, oczekujemy 0
36     printf("Oczekiwano: 0\n");
37     printf("Otrzymano:  %ld\n", licznik); // przy poprawnym uzyciu
        mutexu zawsze 0
38
39     pthread_mutex_destroy(&mutex);
40     return 0;
41 }
```

Listing 1: Kod poprawionej wersji programu z muteksami

3. Output z działania poprawnej wersji

Poniżej przedstawiono zrzut ekranu z terminala prezentujący wynik działania poprawionej wersji programu. Dzięki zastosowaniu muteksów, program za każdym razem daje przewidywalny

wynik, zgodny z oczekiwaniami.



```
PS C:\Users\henry\Desktop\P3> .\race_good.exe
Oczekiwano: 0
Otrzymano: 0
PS C:\Users\henry\Desktop\P3> █
```

Rysunek 1: Wynik działania poprawnej wersji programu

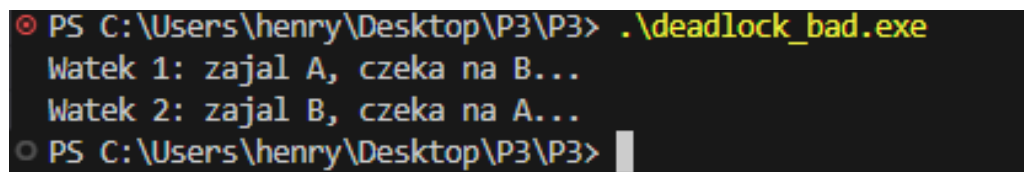
4. Opis: dlaczego wyniki race_bad.c są różne

Wyniki działania programu `race_bad.c` są różne przy każdym uruchomieniu, ponieważ występuje w nim zjawisko wyścigu (ang. *race condition*). Wątki współbieżnie modyfikują tę samą zmienną globalną bez użycia mechanizmów synchronizacji. Operacje inkrementacji i dekrementacji (np. `licznik++`) na poziomie procesora składają się z trzech kroków (pobranie z pamięci, modyfikacja w rejestrze, zapis do pamięci) i nie są atomowe. Powoduje to, że wątki mogą nawzajem nadpisywać swoje wartości, a ostateczny wynik zależy od nieprzewidywalnego przeplotu instrukcji sterowanego przez planistę systemu operacyjnego.

Zadanie 2

1. Zrzut ekranu pokazujący zawieszony program `deadlock_bad.c`

Poniższy zrzut ekranu prezentuje działanie programu `deadlock_bad.c`, w którym doszło do zakleszczenia (deadlock) z powodu niespójnej kolejności blokowania muteksów. Program zawiesza się i nie jest w stanie kontynuować działania.



```
PS C:\Users\henry\Desktop\P3\P3> .\deadlock_bad.exe
Watek 1: zajął A, czeka na B...
Watek 2: zajął B, czeka na A...
PS C:\Users\henry\Desktop\P3\P3> █
```

Rysunek 2: Zawieszony program wykazujący zjawisko zakleszczenia

2. Kod źródłowy wersji z trzema muteksami

Poniższy kod rozwiązuje problem zakleszczenia przy użyciu trzech muteksów. Aby uniknąć deadlocka, w obu wątkach zachowano ścisłą i ustaloną kolejność zajmowania zasobów (A, następnie B, na końcu C).

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 pthread_mutex_t mutex_A;
6 pthread_mutex_t mutex_B;
7 pthread_mutex_t mutex_C; // Dodany trzeci mutex
8
9 void* watek1(void* arg) {
10     pthread_mutex_lock(&mutex_A); // zawsze najpierw A
11     pthread_mutex_lock(&mutex_B); // potem B
12     pthread_mutex_lock(&mutex_C); // na końcu C
```

```

13     printf("Watek 1: zajął wszystkie trzy mutexy (A, B, C)\n");
14
15     // Zwalnianie w odwrotnej kolejności
16     pthread_mutex_unlock(&mutex_C);
17     pthread_mutex_unlock(&mutex_B);
18     pthread_mutex_unlock(&mutex_A);
19     return NULL;
20 }
21
22 void* watek2(void* arg) {
23     pthread_mutex_lock(&mutex_A); // również najpierw A
24     pthread_mutex_lock(&mutex_B); // potem B
25     pthread_mutex_lock(&mutex_C); // na końcu C -- ta sama kolejność!
26     printf("Watek 2: zajął wszystkie trzy mutexy (A, B, C)\n");
27
28     // Zwalnianie w odwrotnej kolejności
29     pthread_mutex_unlock(&mutex_C);
30     pthread_mutex_unlock(&mutex_B);
31     pthread_mutex_unlock(&mutex_A);
32     return NULL;
33 }
34
35 int main(void) {
36     pthread_mutex_init(&mutex_A, NULL);
37     pthread_mutex_init(&mutex_B, NULL);
38     pthread_mutex_init(&mutex_C, NULL); // Inicjalizacja mutexu C
39
40     pthread_t t1, t2;
41     pthread_create(&t1, NULL, watek1, NULL);
42     pthread_create(&t2, NULL, watek2, NULL);
43
44     pthread_join(t1, NULL);
45     pthread_join(t2, NULL);
46
47     pthread_mutex_destroy(&mutex_A);
48     pthread_mutex_destroy(&mutex_B);
49     pthread_mutex_destroy(&mutex_C); // Zniszczenie mutexu C
50
51     printf("Gotowe -- brak deadlocka przy 3 mutexach!\n");
52     return 0;
53 }

```

Listing 2: Poprawna implementacja z trzema mutexami

3. Opis: warunki Coffmana w deadlock_bad.c

W programie `deadlock_bad.c` spełnione są wszystkie cztery warunki Coffmana, co bezpośrednio doprowadziło do zakleszczenia. Zasoby (mutexy) podlegają **wzajemnemu wykluczaniu** i nie ma możliwości ich **wywłaszczenia** przez system operacyjny. Ponadto zachodzi **przetrzymywanie i oczekiwanie** (wątek trzyma jeden zasób, czekając na kolejny) oraz **cykliczne oczekiwanie** (wątek 1 czeka na mutex B trzymany przez wątek 2, a wątek 2 czeka na mutex A trzymany przez wątek 1).

Zadanie 3

1. Kod źródłowy programu z dwoma konsumentami

Zgodnie z poleceniem, program został zmodyfikowany: dodano drugiego konsumenta (każdy pobiera dokładnie połowę z wyprodukowanych elementów), a także zaimplementowano licznik śledzący aktualną liczbę elementów w buforze.

```
1  #include <pthread.h>
2  #include <semaphore.h>
3  #include <stdio.h>
4  #include <unistd.h>
5
6  #define PRODUCENTOW 3
7  #define KONSUMENTOW 2
8  #define BUFOR_MAX 5
9
10 // Klasyczny problem producent-konsument z semaforem
11 sem_t sem_wolne; // ile jest wolnych miejsc w buforze
12 sem_t sem_zajete; // ile jest elementow do pobrania
13 pthread_mutex_t mutex_bufor;
14
15 int bufor[BUFOR_MAX];
16 int wpisz = 0, odczytaj = 0;
17 int elementow_w_buforze = 0; // Dodany licznik elementow
18
19 void* producent(void* arg) {
20     int id = *(int*)arg;
21
22     for (int i = 0; i < 4; i++) {
23         int element = id * 10 + i;
24
25         sem_wait(&sem_wolne); // czekaj na wolne miejsce
26         pthread_mutex_lock(&mutex_bufor);
27
28         bufor[wpisz] = element;
29         wpisz = (wpisz + 1) % BUFOR_MAX;
30         elementow_w_buforze++; // Zwiększ licznik po dodaniu
31
32         printf("[P%d] wpisano: %d | W buforze: %d\n", id, element,
33             elementow_w_buforze);
34
35         pthread_mutex_unlock(&mutex_bufor);
36         sem_post(&sem_zajete); // zasygnalizuj nowy element
37         usleep(100000);
38     }
39     return NULL;
40 }
41
42 void* konsument(void* arg) {
43     int id = *(int*)arg;
44     // Calkowita liczba elementow to PRODUCENTOW * 4.
45     // Dzielimy ja przez liczbe konsumentow, wiec kazdy pobierze polowe
46     // (po 6 elementow).
47     int ile_pobrac = (PRODUCENTOW * 4) / KONSUMENTOW;
```

```

46
47     for (int i = 0; i < ile_pobrac; i++) {
48         sem_wait(&sem_zajete);           // czekaj az bedzie co pobierac
49         pthread_mutex_lock(&mutex_bufor);
50
51         int element = bufor[odczytaj];
52         odczytaj = (odczytaj + 1) % BUFOR_MAX;
53         elementow_w_buforze--; // Zmniejsz licznik po pobraniu
54
55         printf("[K%d] pobrano: %d | W buforze: %d\n", id, element,
56             elementow_w_buforze);
57
58         pthread_mutex_unlock(&mutex_bufor);
59         sem_post(&sem_wolne);           // zwolnij miejsce w buforze
60         usleep(150000);
61     }
62     return NULL;
63 }
64
65 int main(void) {
66     sem_init(&sem_wolne, 0, BUFOR_MAX); // na poczatku bufor pusty
67     sem_init(&sem_zajete, 0, 0);        // brak elementow
68     pthread_mutex_init(&mutex_bufor, NULL);
69
70     pthread_t prod[PRODUCENTOW], kons[KONSUMENTOW];
71     int ids_prod[PRODUCENTOW];
72     int ids_kons[KONSUMENTOW];
73
74     // Tworzenie watkow producentow
75     for (int i = 0; i < PRODUCENTOW; i++) {
76         ids_prod[i] = i + 1;
77         pthread_create(&prod[i], NULL, producent, &ids_prod[i]);
78     }
79
80     // Tworzenie watkow konsumentow
81     for (int i = 0; i < KONSUMENTOW; i++) {
82         ids_kons[i] = i + 1;
83         pthread_create(&kons[i], NULL, konsument, &ids_kons[i]);
84     }
85
86     // Oczekiwanie na zakonczenie watkow producentow
87     for (int i = 0; i < PRODUCENTOW; i++) {
88         pthread_join(prod[i], NULL);
89     }
90
91     // Oczekiwanie na zakonczenie watkow konsumentow
92     for (int i = 0; i < KONSUMENTOW; i++) {
93         pthread_join(kons[i], NULL);
94     }
95
96     sem_destroy(&sem_wolne);
97     sem_destroy(&sem_zajete);
98     pthread_mutex_destroy(&mutex_bufor);

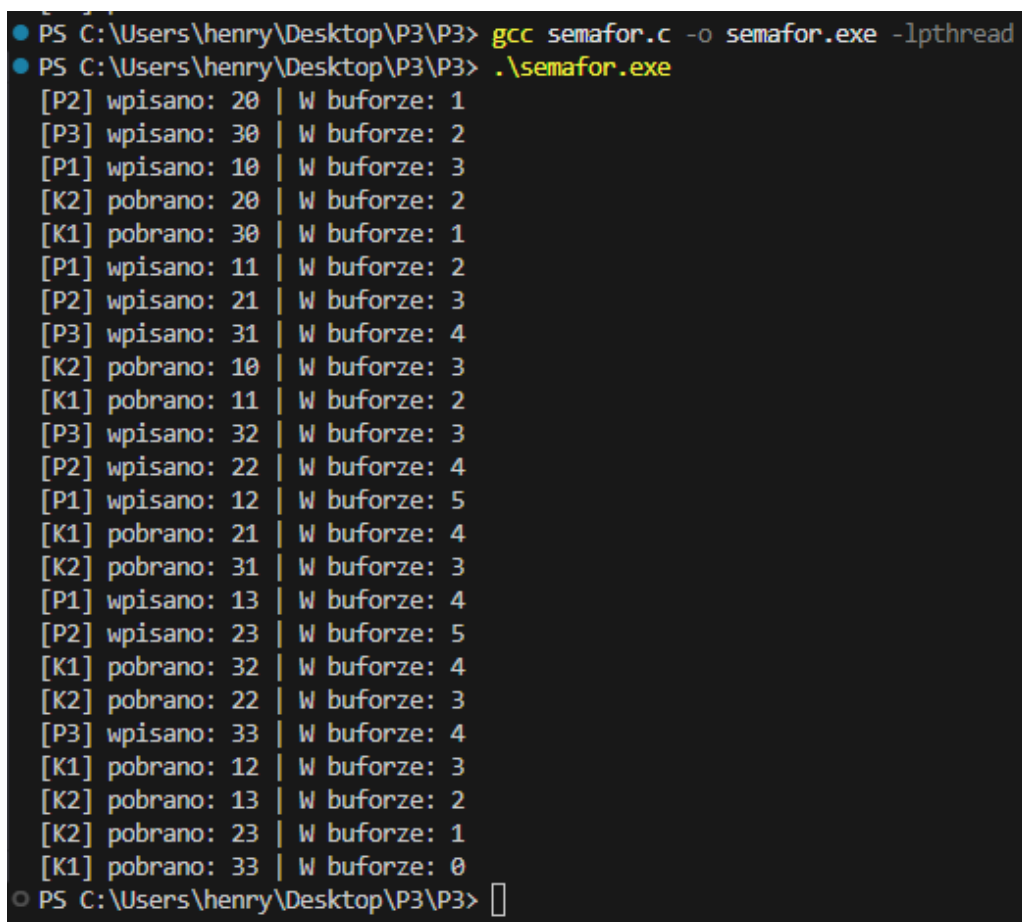
```

```
98     return 0;
99 }
```

Listing 3: Problem producenta-konsumenta (3 producentów, 2 konsumentów, licznik bufora)

2. Output z działania programu

Poniższy zrzut ekranu prezentuje poprawną współpracę trzech producentów i dwóch konsumentów. Widoczne jest, że pojemność bufora nigdy nie przekracza zdefiniowanej wielkości (BUFOR_MAX = 5), a konsumenci dzielą się pracą.



```
PS C:\Users\henry\Desktop\P3\P3> gcc semafor.c -o semafor.exe -lpthread
PS C:\Users\henry\Desktop\P3\P3> .\semafor.exe
[P2] wpisano: 20 | W buforze: 1
[P3] wpisano: 30 | W buforze: 2
[P1] wpisano: 10 | W buforze: 3
[K2] pobrano: 20 | W buforze: 2
[K1] pobrano: 30 | W buforze: 1
[P1] wpisano: 11 | W buforze: 2
[P2] wpisano: 21 | W buforze: 3
[P3] wpisano: 31 | W buforze: 4
[K2] pobrano: 10 | W buforze: 3
[K1] pobrano: 11 | W buforze: 2
[P3] wpisano: 32 | W buforze: 3
[P2] wpisano: 22 | W buforze: 4
[P1] wpisano: 12 | W buforze: 5
[K1] pobrano: 21 | W buforze: 4
[K2] pobrano: 31 | W buforze: 3
[P1] wpisano: 13 | W buforze: 4
[P2] wpisano: 23 | W buforze: 5
[K1] pobrano: 32 | W buforze: 4
[K2] pobrano: 22 | W buforze: 3
[P3] wpisano: 33 | W buforze: 4
[K1] pobrano: 12 | W buforze: 3
[K2] pobrano: 13 | W buforze: 2
[K2] pobrano: 23 | W buforze: 1
[K1] pobrano: 33 | W buforze: 0
PS C:\Users\henry\Desktop\P3\P3> 
```

Rysunek 3: Wynik działania programu z dwoma konsumentami

3. Opis: dlaczego potrzebny jest zarówno mutex, jak i semafor?

Do ochrony bufora potrzebne są oba mechanizmy, ponieważ pełnią one różne role. Semafor służy do **synchronizacji stanu** – usypia wątki, gdy bufor jest pełny (producenci) lub pusty (konsumenti), tym samym eliminując problem pożerającego zasoby procesora aktywnego oczekiwania (tzw. *busy waiting*). Z kolei mutex zapewnia **wzajemne wykluczanie** podczas modyfikacji wspólnych zmiennych (samej tablicy bufora, indeksów zapisu/odczytu oraz licznika), co zapobiega nadpisywaniu się danych przez równoległe działające wątki (zjawisku wyścigu). Sam mutex nie wystarczyłby, ponieważ nie posiada natywnego mechanizmu blokującego (usypiającego) wątek na podstawie określonego warunku logicznego (np. "oczekuj, aż w buforze pojawi się wolne miejsce").

III. Pytania kontrolne

1. Czym jest *race condition*? Podaj przykład z życia codziennego (poza informatyką) który ilustruje ten problem.

Zjawisko wyścigu (ang. *race condition*) występuje, gdy końcowy wynik operacji zależy od nieprzewidywalnego przeplotu działań wykonywanych współbieżnie przez kilka podmiotów na współdzielonym zasobie. Przykładem z życia codziennego jest sytuacja, w której dwoje domowników zauważa brak mleka w lodówce i niezależnie od siebie udaje się do sklepu, by je kupić, nie komunikując się ze sobą. W efekcie, zamiast jednego kartonu, w domu pojawiają się dwa (nadpisanie stanu oczekiwanego), co jest błędem wynikającym z braku synchronizacji i wzajemnego wykluczania (np. poprzez zostawienie kartki "poszedłem po mleko").

2. Dlaczego operacja `licznik++` nie jest atomowa? Z ilu kroków maszynowych się składa?

Operacja inkrementacji (`licznik++`) nie jest atomowa, ponieważ na poziomie architektury procesora nie jest wykonywana jako pojedyncza, niepodzielna instrukcja. Składa się ona z trzech odrębnych kroków maszynowych: pobrania obecnej wartości zmiennej z pamięci operacyjnej do rejestru procesora, zwiększenia tej wartości o jeden wewnątrz układu arytmetyczno-logicznego (ALU) oraz zapisania nowego wyniku z rejestru z powrotem do pamięci RAM. Jeśli w trakcie tych trzech kroków system operacyjny przełączy kontekst na inny wątek, nastąpi nadpisanie wyników pracy, co doprowadzi do zjawiska wyścigu.

3. Co to jest sekcja krytyczna i jakie warunki powinna spełniać (wzajemne wykluczanie, postęp, ograniczone oczekiwanie)?

Sekcja krytyczna to fragment kodu, w którym proces lub wątek uzyskuje dostęp do współdzielonych zasobów (np. zmiennych globalnych, plików, urządzeń zewnętrznych), co może prowadzić do niespójności danych w przypadku braku synchronizacji. Aby synchronizacja była uznana za w pełni poprawną, mechanizm chroniący sekcję musi spełniać trzy warunki. Są to: **wzajemne wykluczanie** (tylko jeden wątek może w niej przebywać w danym czasie), **postęp** (jeśli nikt nie jest w sekcji, a wątki chcą wejść, decyzja o wpuszczeniu jednego z nich nie może być odkładana w nieskończoność) oraz **ograniczone oczekiwanie** (musi istnieć górny limit czasu lub liczby wejść innych wątków, zanim dany wątek zostanie dopuszczony do sekcji, co zapobiega jego zagłodzeniu).

4. Wymień cztery warunki Coffmana konieczne do powstania deadlocka. Wyjaśnij jak wyeliminowanie jednego z nich zapobiega zakleszczeniu.

Do powstania zakleszczenia niezbędne jest jednoczesne wystąpienie czterech warunków Coffmana: wzajemnego wykluczania, przetrzymywania i oczekiwania na zasoby, braku możliwości wywłaszczenia zasobu (musi być oddany dobrowolnie) oraz cyklicznego oczekiwania wątków na siebie nawzajem. Wyeliminowanie chociażby jednego z nich (np. złamanie warunku cyklicznego oczekiwania poprzez narzucenie ścisłej, globalnej kolejności blokowania muteksów, jak w zadaniu 2) sprawia, że łańcuch zależności zostaje bezpowrotnie przerwany, a deadlock staje się logicznie niemożliwy do wystąpienia.

5. Czym różni się deadlock od starvation? Podaj przykład każdego z nich.

Zakleszczenie (deadlock) to sytuacja bez wyjścia, w której grupa wątków jest trwale zablokowana, ponieważ każdy z nich czeka na zasób przetrzymywany przez innego członka tej samej grupy (np. na skrzyżowaniu równorzędnym cztery samochody blokują się nawzajem i żaden nie może ruszyć). Głodzenie (starvation) występuje z kolei wtedy, gdy wątek jest technicznie gotowy do działania, ale w nieskończoność odmawia mu się dostępu do procesora lub zasobu, ponieważ system nieustannie faworyzuje inne wątki (np. proces o bardzo niskim priorytecie).

nigdy nie dostaje czasu procesora, ponieważ do kolejki ciągle trafiają nowe procesy o wyższym priorytecie).

6. Czym różni się semafor od mutexa? Kiedy użyjesz semafora zamiast mutexa?

Mutex służy wyłącznie do wzajemnego wykluczania i posiada ścisłą koncepcję "własności" – tylko ten wątek, który zablokował mutex, ma prawo go odblokować. Semafor jest z kolei mechanizmem sygnalizacyjnym opartym na liczniku, nie ma przypisanego "właściciela" (jeden wątek może obniżyć jego wartość, a zupełnie inny ją podnieść) i może dopuszczać do zasobu więcej niż jeden wątek. Semafora użyjemy zamiast mutexu wtedy, gdy chcemy zarządzać pulą identycznych zasobów (np. 5 wolnych miejsc w buforze) lub gdy potrzebujemy zsynchronizować kolejność wykonywania zadań między różnymi wątkami (np. wątek B musi czekać na sygnał od wątku A).

7. Co się stanie jeśli zapomnisz wywołać `pthread_mutex_unlock` po wyjściu z sekcji krytycznej?

Jeśli po opuszczeniu sekcji krytycznej programista zapomni wywołać funkcję odblokowującą `pthread_mutex_unlock`, dany mutex pozostanie w stanie zamkniętym na zawsze. Każdy kolejny wątek, który spróbuje wejść do tej sekcji krytycznej i wywoła `pthread_mutex_lock`, zostanie trwale uśpiony przez system operacyjny, czekając na zasób, który nigdy się nie zwolni. W efekcie doprowadzi to do całkowitego zawieszenia się programu (bądź jego kluczowych funkcji), trwale uniemożliwiając mu realizację powierzonych zadań.

8. Na czym polega problem producenta–konsumenta? Jaką rolę pełnią w jego rozwiązaniu dwa semafony?

Problem producenta i konsumenta polega na bezpiecznej oraz bezbłędnej wymianie danych między wątkami generującymi informacje (producentami) a wątkami je przetwarzającymi (konsumentami) przy użyciu wspólnego bufora o ograniczonej pojemności. Dwa semafony odgrywają w jego rozwiązaniu kluczową rolę synchronizacyjną, zapobiegając marnującemu moc procesora aktywnemu oczekiwaniu (*busy waiting*): semafor śledzący "wolne miejsca" usypia producentów, gdy bufor jest pełny, natomiast semafor śledzący "zajęte miejsca" usypia konsumentów, gdy w buforze nie ma nic do pobrania.